UI POLICY

1. Introduction

User Interface (UI) Policy is a set of rules and conditions used to control the appearance and behavior of fields and elements in a user interface. It helps determine how fields should behave when a user interacts with a form or application. UI policies are commonly used in web applications, enterprise software, and platforms where forms are used to collect and manage information.

A well-designed UI policy improves the usability of an application by showing only the information that is necessary at a particular time. For example, a form may contain several fields, but some fields may need to be displayed only when a particular option is selected. A UI policy can automatically control these fields without requiring the user to perform additional actions.

UI policies can be used to make fields visible or hidden, mandatory or optional, and enabled or disabled. This provides a more organized interface and reduces unnecessary information on the screen. It also helps users complete forms correctly and efficiently.

---

2. Objectives of UI Policy

The main objective of a UI policy is to provide a dynamic and user-friendly interface. Instead of displaying all fields and options at the same time, the interface can respond according to the user's actions.

The important objectives of UI policy include:

1. Improving User Experience: UI policies make applications easier to understand and use.

2. Controlling Field Visibility: Fields can be shown or hidden depending on specific conditions.

3. Managing Mandatory Fields: A field can be automatically made mandatory when certain conditions are satisfied.

4. Controlling Field Availability: Fields can be enabled or disabled according to the user's selections.

5. Reducing User Errors: By displaying only relevant fields and requiring important information, UI policies can reduce incorrect data entry.

6. Simplifying Forms: Unnecessary fields can be hidden until they are required.

7. Improving Data Quality: Mandatory and controlled fields help ensure that important information is properly entered.

---

3. Components of UI Policy

A UI policy generally consists of several important components. These components work together to determine how the interface should respond to a particular condition.

3.1 Condition

The condition determines when the UI policy should be applied. It can depend on the value selected or entered by the user.

For example, consider a student registration form. If the user selects "Hosteller" as the accommodation type, additional fields such as hostel name and room number may become necessary.

3.2 UI Policy Action

A UI policy action defines what should happen when the condition is satisfied. Common actions include changing field visibility, making fields mandatory, or making fields read-only.

3.3 Visibility

Visibility determines whether a particular field should be displayed to the user. A field can be hidden when it is not relevant and displayed when it becomes necessary**.**

3.4 Mandatory State

A field can be made mandatory when certain conditions are met. This ensures that users provide important information before submitting a form.

3.5 Read-Only State

A UI policy can also make a field read-only. Users can view the information but cannot modify it.

---

4. Types of UI Policy

UI policies can be implemented in different ways depending on the requirements of an application.

4.1 Visibility Policy

A visibility policy controls whether a field or interface element is displayed. It is useful when certain information is relevant only in specific situations.

For example, if a student selects "Yes" for having transportation facilities, additional transportation-related fields can be displayed.

4.2 Mandatory Policy

A mandatory policy requires users to enter information into a particular field. This is useful for collecting essential information such as name, email address, phone number, or identification number.

4.3 Read-Only Policy

A read-only policy prevents users from modifying a particular field. It is useful for information that should be displayed but should not be changed by ordinary users.

4.4 Enable or Disable Policy

This type of policy controls whether users can interact with a particular field or control. A field may remain disabled until another required selection is made.

---

5. Working of UI Policy

The working of a UI policy can be explained through a simple sequence.

First, the system displays a form containing different fields. The user enters information or selects an option. The application checks the conditions associated with the UI policy.

If the condition is satisfied, the corresponding UI policy action is executed. The system may then show a hidden field, make a field mandatory, disable a field, or make a field read-only.

If the condition is not satisfied, the system applies the alternative behavior defined by the application. This process allows the interface to change dynamically according to user interaction.

For example, consider an employee form. If the employee selects "Experienced" as their employment status, an additional field called "Previous Company" can be displayed. If "Fresher" is selected, the field can remain hidden.

---

6. Advantages of UI Policy

UI policies provide several advantages in application development.

6.1 Better User Experience

UI policies create a cleaner and more organized interface. Users can focus on the fields that are relevant to their current task.

6.2 Reduced Complexity

Forms containing many fields can become difficult to understand. UI policies help reduce complexity by hiding unnecessary fields.

6.3 Improved Data Accuracy

Mandatory fields and controlled input help users provide complete and accurate information**.**

6.4 Reduced User Errors

When irrelevant fields are hidden or disabled, users are less likely to enter incorrect information.

6.5 Dynamic Interface

UI policies allow the interface to respond automatically to user actions. This makes applications more interactive.

6.6 Improved Productivity

Users can complete forms more quickly because they do not have to deal with unnecessary fields.

6.7 Consistent Behavior

UI policies help maintain consistent interface behavior throughout an application.

---

7. UI Policy in Web Applications

UI policies are particularly useful in web applications because web forms often contain many fields. A form can dynamically change according to the user's selections.

For example, an online college admission form may ask whether the student requires hostel accommodation. If the student selects "Yes," hostel-related fields can appear automatically. If the student selects "No," those fields can remain hidden.

Similarly, an online service request form may display different fields depending on the type of service selected. This approach keeps the interface simple while still allowing the application to collect detailed information when required.

UI policies can therefore improve both the design and functionality of web-based forms.

---

8. UI Policy and User Experience

User experience is an important part of modern software development. An application may have powerful functionality, but users may find it difficult to use if the interface is confusing.

UI policies help solve this problem by presenting information according to the user's current needs. Instead of showing every possible option, the application can display only the relevant choices.

For example, in a school management system, an administrator may select a particular student category. Based on that selection, additional fields related to the category can automatically appear.

This creates a more understandable and efficient interface. It also helps users complete tasks with fewer unnecessary steps.

---

9. UI Policy and Data Validation

UI policy and data validation can work together to improve data quality. UI policies control the behavior of fields, while validation ensures that the entered information satisfies the required rules.

For example, a UI policy may make the phone number field mandatory when a particular option is selected. Validation can then check whether the entered phone number follows the expected format.

Similarly, a UI policy may display an email field when required, while validation checks whether a valid email address has been entered.

Therefore, combining UI policies with proper validation provides a better mechanism for collecting reliable information.

---

10. Example of UI Policy

Consider a Student Management System.

The registration form contains the following fields:

- Student Name
- Admission Number
- Student Type
- Hostel Required
- Hostel Name
- Room Number
- Parent Phone Number

Suppose the user selects "Yes" for the Hostel Required field. The UI policy can automatically display the Hostel Name and Room Number fields.

If the user selects "No," those fields can be hidden.

The policy can also make Hostel Name and Room Number mandatory when the student requires hostel accommodation.

This example demonstrates how UI policies can dynamically control the interface according to user input.

---

11. Difference Between UI Policy and UI Design

UI policy and UI design are related but different concepts.

UI Design focuses on the overall visual appearance and structure of an application. It includes colors, fonts, layouts, buttons, icons, spacing, and navigation.

UI Policy focuses on the behavior of interface elements under particular conditions. It determines whether fields should be visible, mandatory, read-only, enabled, or disabled.

For example, choosing the color and size of a button is part of UI design, while deciding whether that button should be enabled only after completing required fields is related to interface behavior and policy.

Both are important for creating an effective application.

---

12. Best Practices for UI Policy

The following practices can help in creating effective UI policies:

12.1 Keep Conditions Simple

Conditions should be easy to understand and maintain.

12.2 Show Only Relevant Information

Avoid displaying unnecessary fields to users.

12.3 Use Mandatory Fields Carefully

Only important fields should be made mandatory. Making too many fields mandatory can make forms difficult to complete.

12.4 Provide Clear Feedback

When a field becomes mandatory or changes its state, users should be able to understand what is expected from them.

12.5 Maintain Consistency

Similar conditions should produce similar interface behavior throughout the application.

12.6 Test Different Scenarios

UI policies should be tested with different combinations of user inputs to ensure that the interface behaves correctly.

---

13. Limitations of UI Policy

Although UI policies provide many benefits, they also have some limitations.

Complex UI policies can become difficult to manage when an application contains a large number of conditions. Poorly designed policies may also cause unexpected changes in the interface.

If too many fields are dynamically hidden or displayed, users may become confused. Therefore, UI policies should be designed carefully.

UI policies generally control the behavior of the interface, but they should not be considered a replacement for server-side security or data validation. Important business rules and security checks should also be implemented at the appropriate backend level.

---

14. Applications of UI Policy

UI policies are used in many types of applications.

14.1 Educational Systems

They can control student registration, examination, attendance, and fee-related forms.

14.2 Banking Applications

They can display additional fields depending on the type of account or transaction selected.

14.3 Healthcare Applications

They can display relevant fields based on patient information and selected services.

14.4 E-Commerce Applications

They can change available options based on product type, delivery method, or payment method.

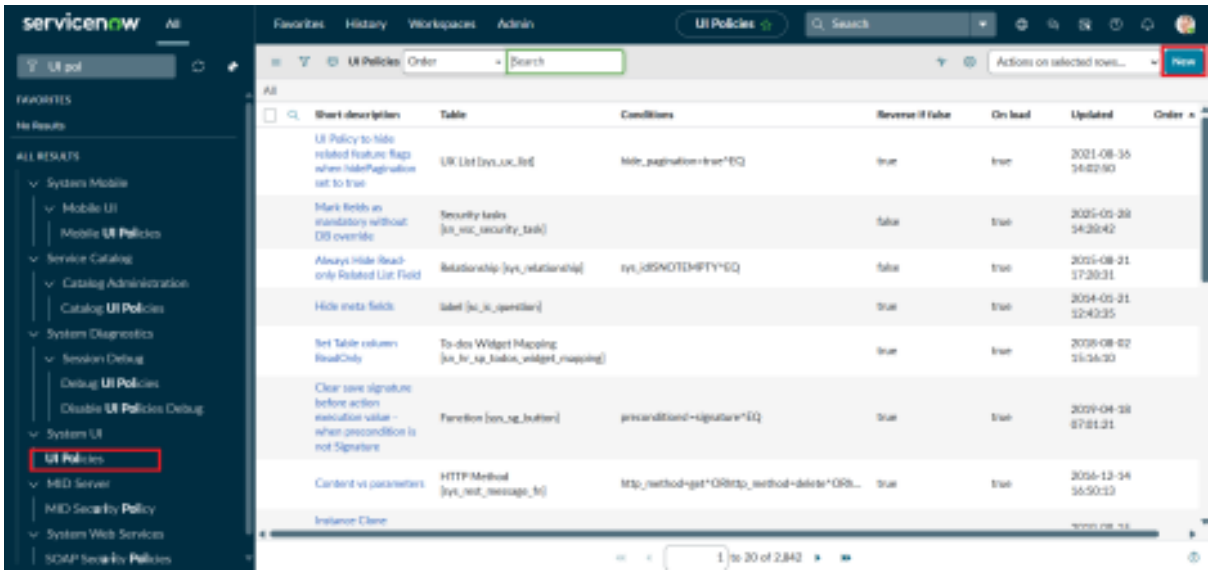
14.5 Employee Management Systems

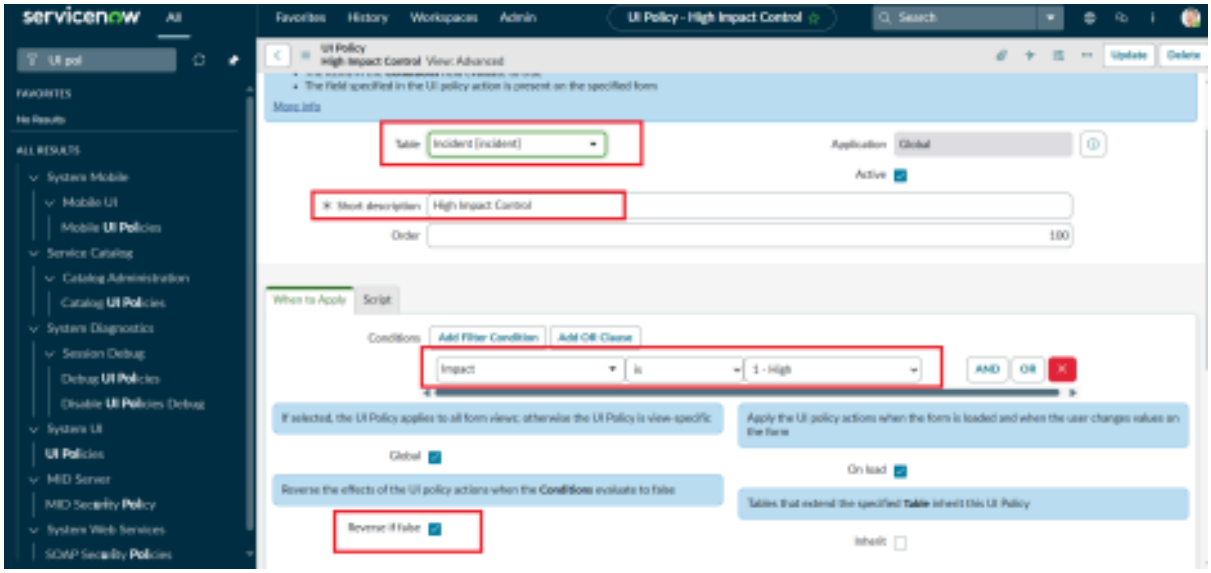
They can control fields related to employee type, department, job role, and previous experience**.**

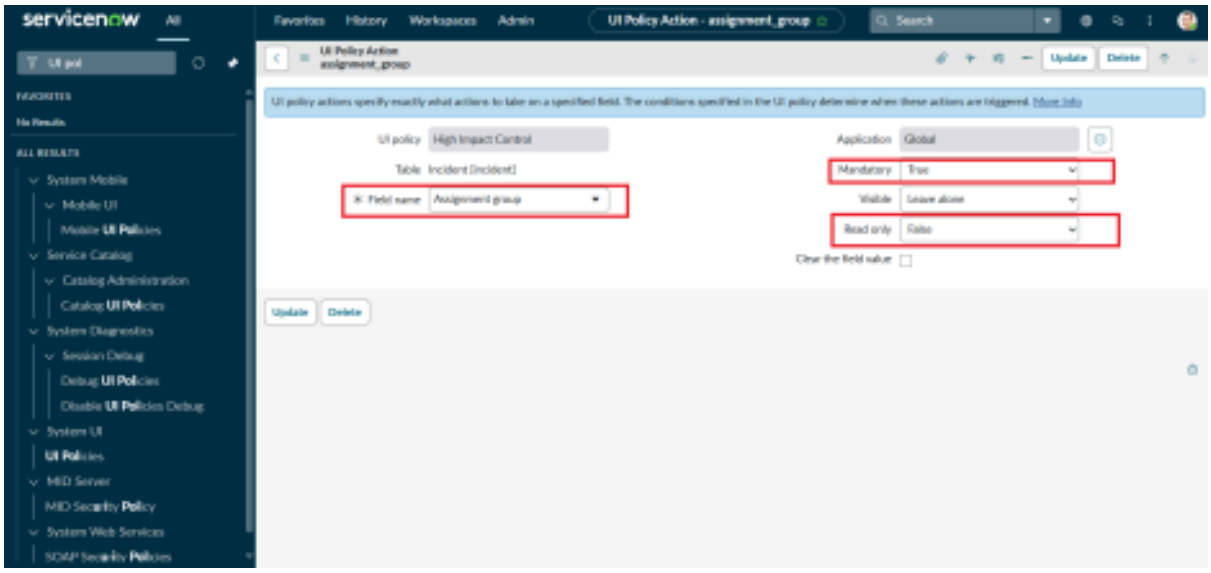
14.6 Customer Service Applications

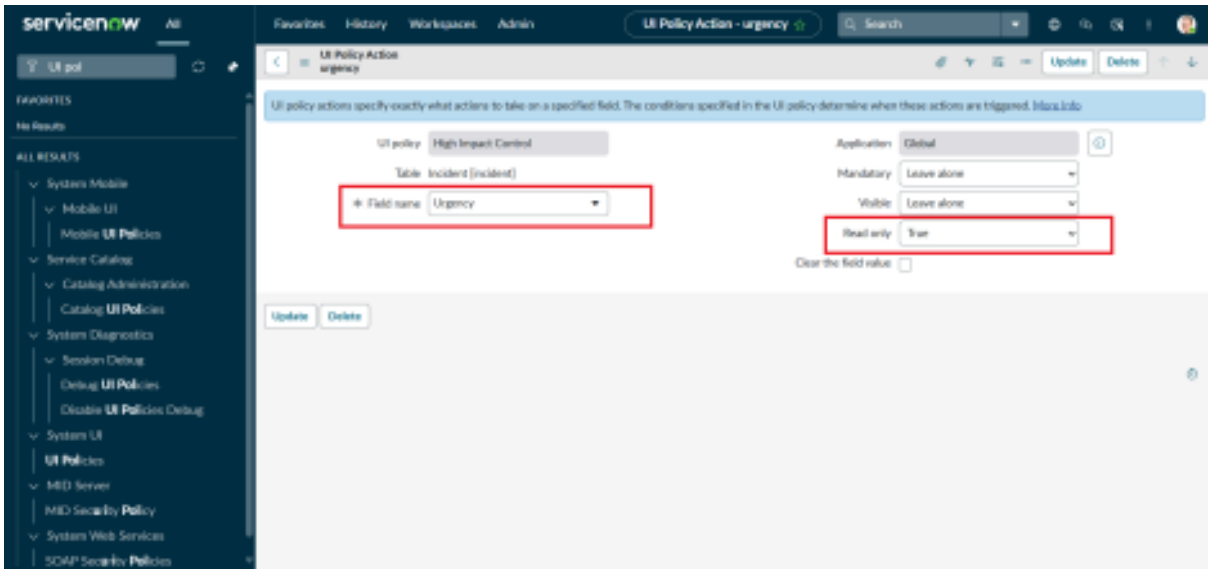
They can display different fields depending on the type of complaint or service request.

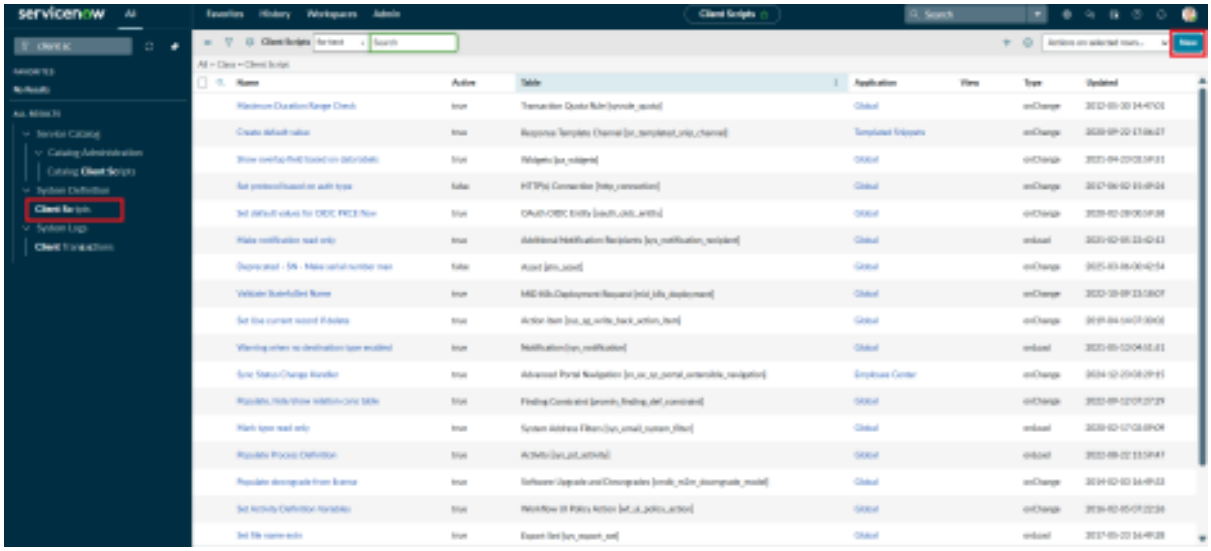
15.Screenshots

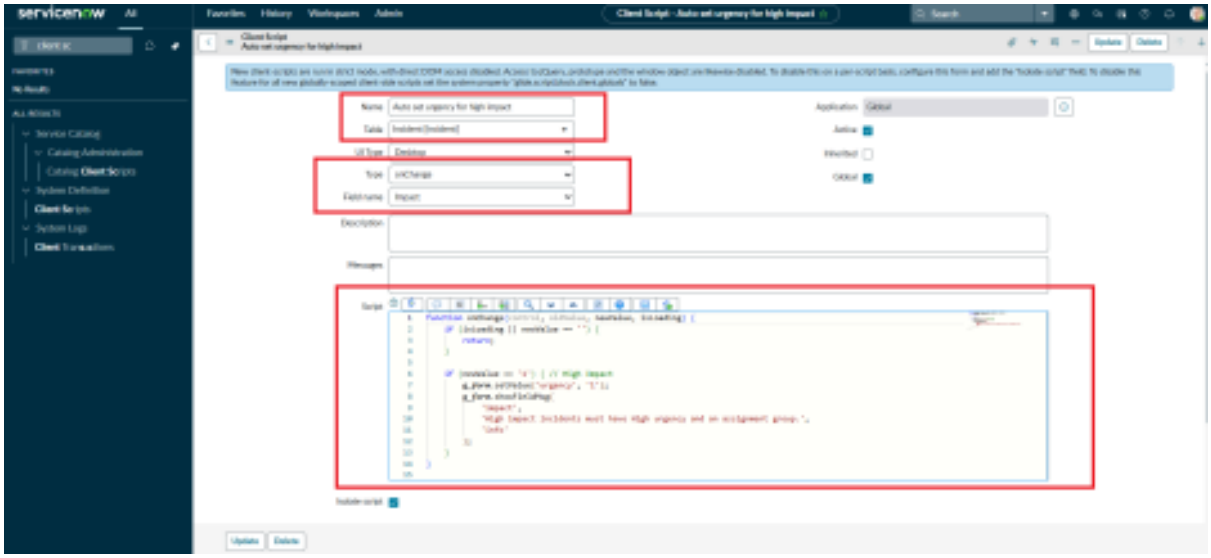


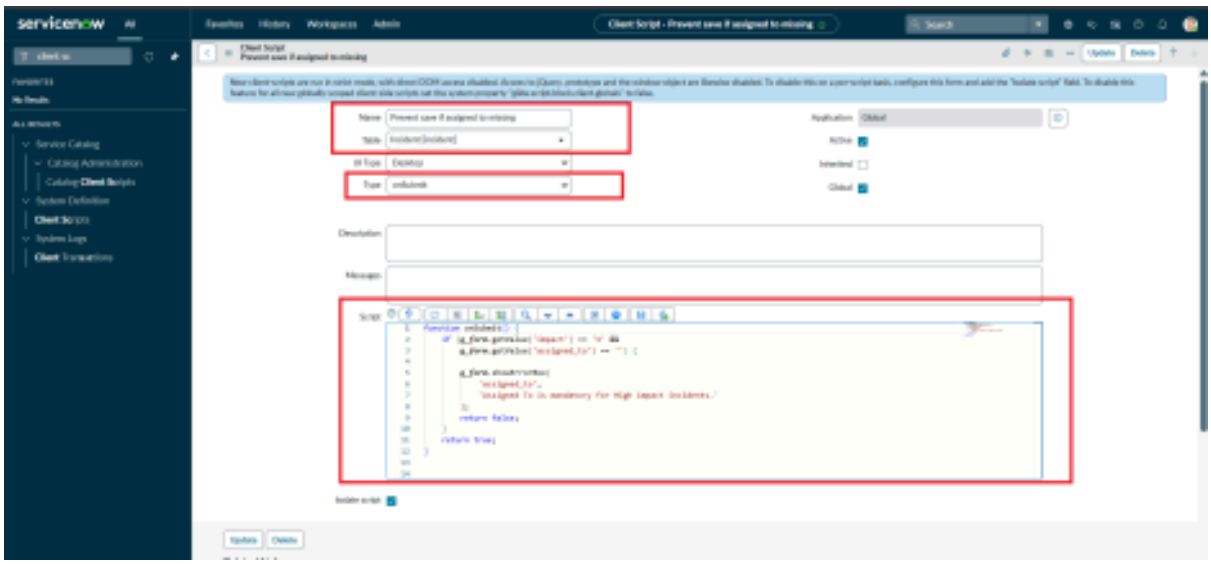

servicenow
Favorites
History
Workspaces
Admin
UI Policy - High Impact Control
Search
UI Policy
High Impact Control View: Advanced
Update
Delete
More info
Table
Incident [incident]
Application
Global
Active
Short description
High Impact Control
Order
100
When to Apply
Script
Conditions
Add Filter Condition
Add OR Clause
Impact
is
1 - High
AND
OR
If selected, the UI Policy applies to all form views; otherwise the UI Policy is view-specific
Apply the UI policy actions when the form is loaded and when the user changes values on the form
Global
On load
Reverse the effects of the UI policy actions when the Conditions evaluate to false
Tables that extend the specified Table inherit this UI Policy
Reverse if false
Inherit

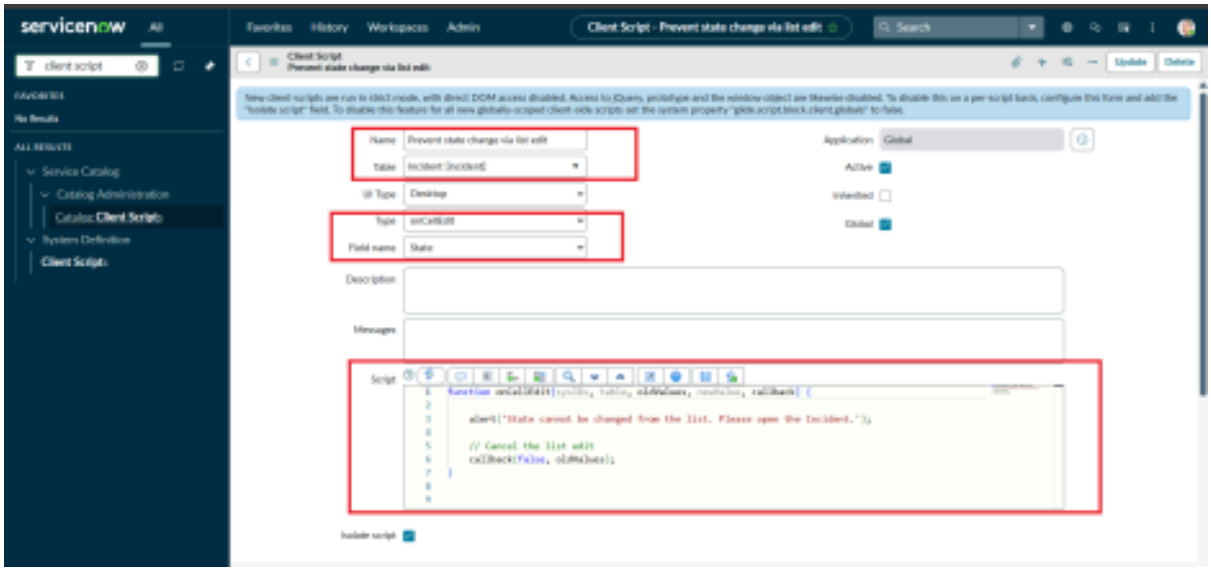

servicenow
Favorites
History
Workspaces
Admin
UI Policy Action - assignment_group
Search
UI Policy Action
assignment_group
Update
Delete
UI policy actions specify exactly what actions to take on a specified field. The conditions specified in the UI policy determine when these actions are triggered. More info
UI policy
High Impact Control
Application
Global
Table
Incident [incident]
Mandatory
True
Field name
Assignment group
Visible
Leave alone
Read only
False
Clear the field value
Update
Delete

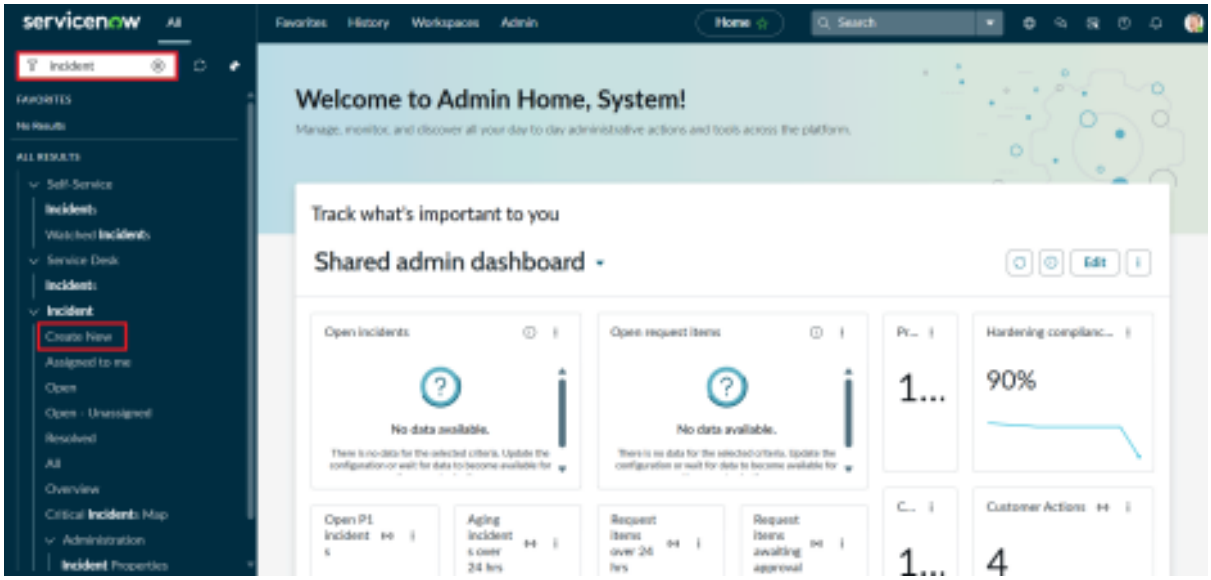

servicenow
Favorites
History
Workspaces
Admin
UI Policy Action - urgency
Search
UI Policy Action
urgency
Update
Delete
UI policy actions specify exactly what actions to take on a specified field. The conditions specified in the UI policy determine when these actions are triggered. More info
UI policy
High Impact Control
Application
Global
Table
Incident [incident]
Mandatory
Leave alone
Field name
Urgency
Visible
Leave alone
Read only
True
Clear the field value
Update
Delete

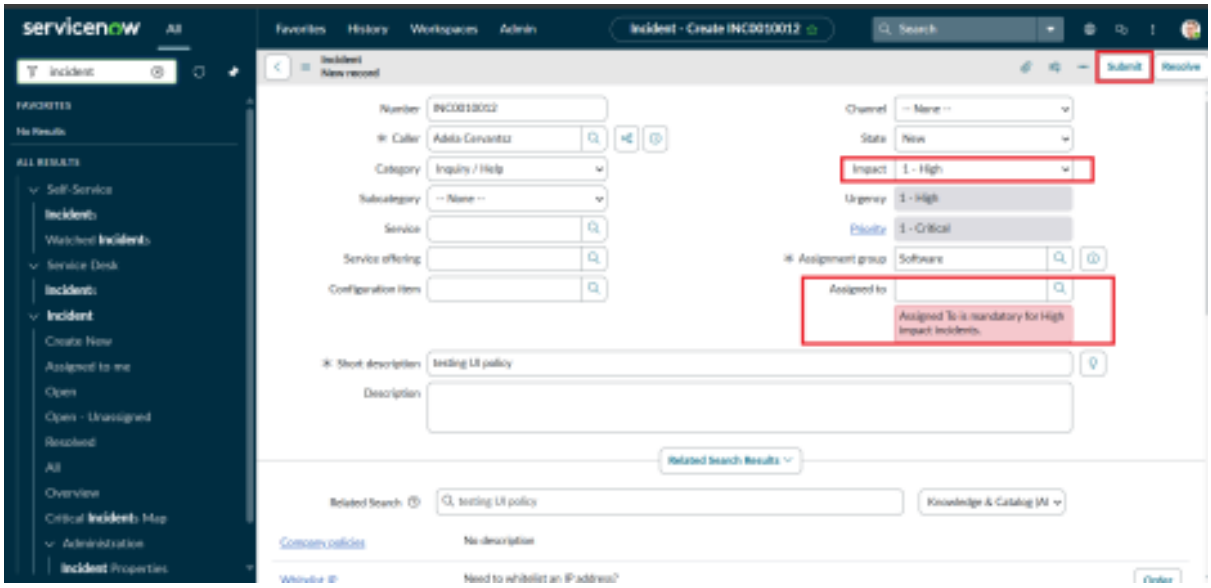


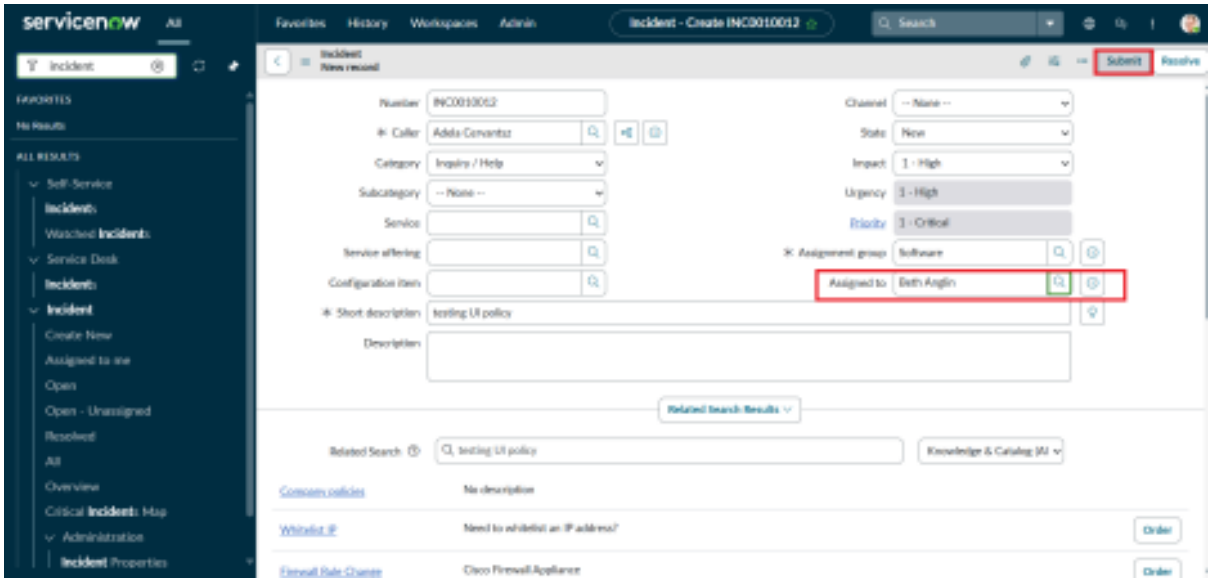


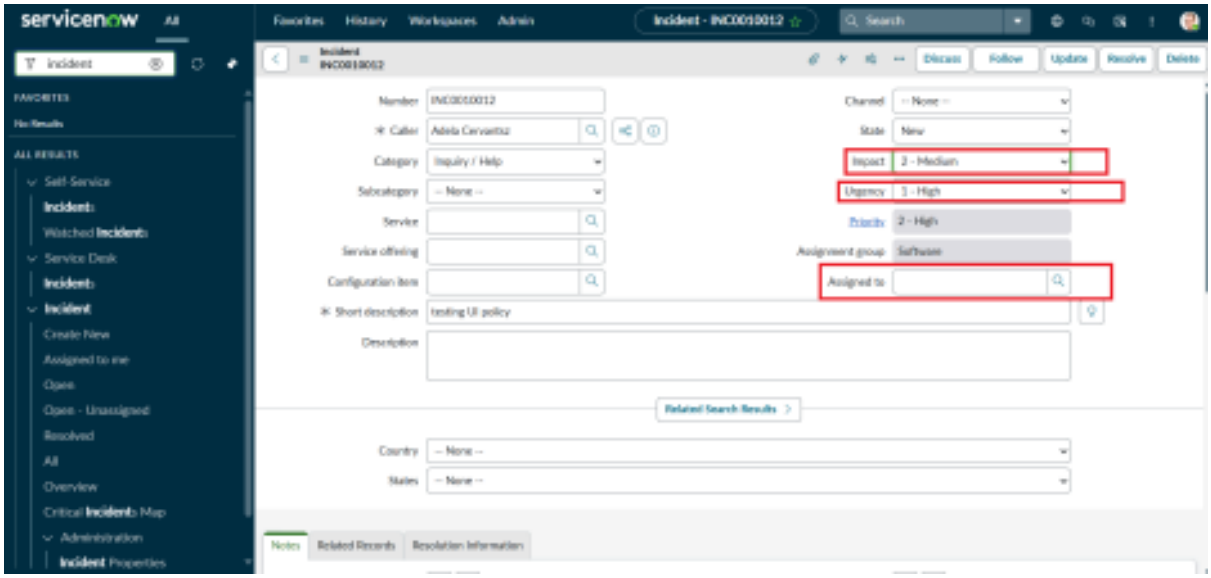


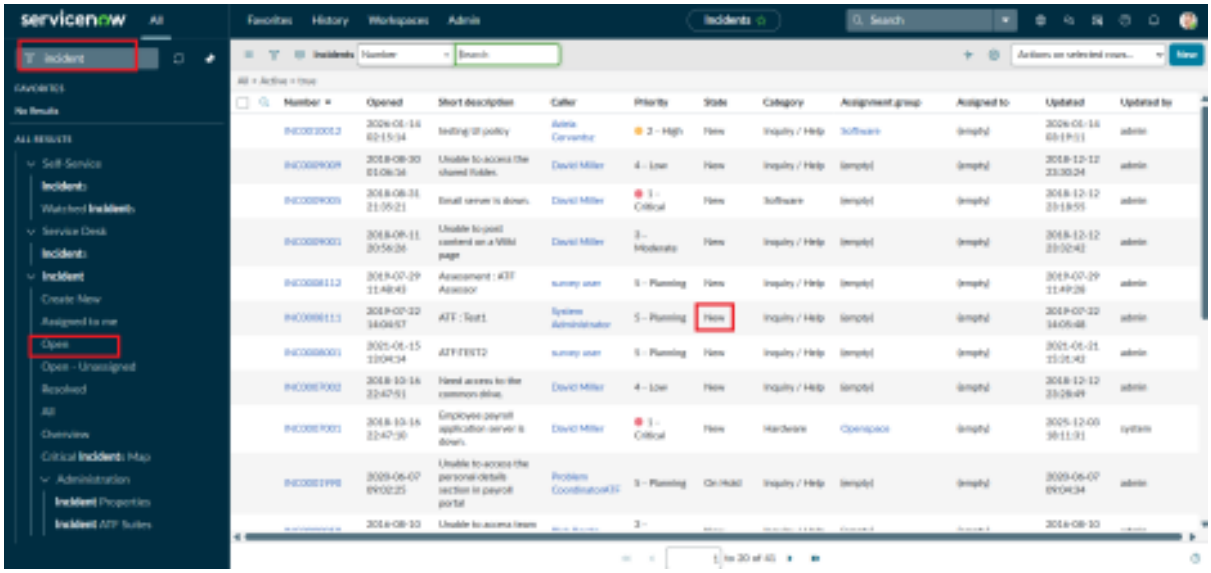

servicenow
All
Favorites
History
Workspaces
Admin
Client Script - Prevent state change via list edit
client script
Service Catalog
Catalog Administration
Catalog Client Scripts
System Definition
Client Scripts
Name
Prevent state change via list edit
Table
Incident [incident]
Type
Desktop
Type
onCellEdit
Field name
State
Application
Global
Active
Inherited
Global
Description
Messages
Script
function onCellEdit(sysId, table, oldValues, newValue, callback) {
alert("State cannot be changed from the list. Please open the Incident.");
// Cancel the list edit
callback(false, oldValues);
}
Isolate script

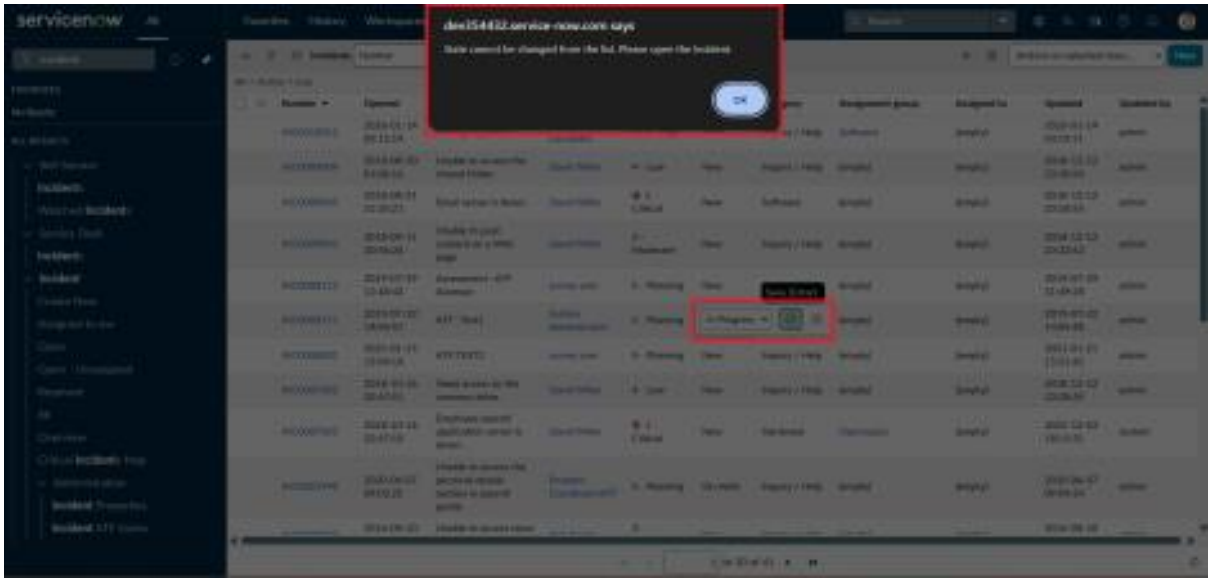

servicenow
All
Favorites
History
Workspaces
Admin
Home
Search
incident
FAVORITES
No Results
ALL RESULTS
Self-Service
Incidents
Watched Incidents
Service Desk
Incidents
Incident
Create New
Assigned to me
Open
Open - Unassigned
Resolved
All
Overview
Critical Incidents Map
Administration
Incident Properties
Welcome to Admin Home, System!
Manage, monitor, and discover all your day to day administrative actions and tools across the platform.
Track what's important to you
Shared admin dashboard
Edit
Open incidents
No data available.
Open request items
No data available.
Pr...
1...
Hardening compliance...
90%
Open P1 Incident s
Aging Incident s over 24 hrs
Request items over 26 hrs
Request items awaiting approval
C...
1...
Customer Actions
4

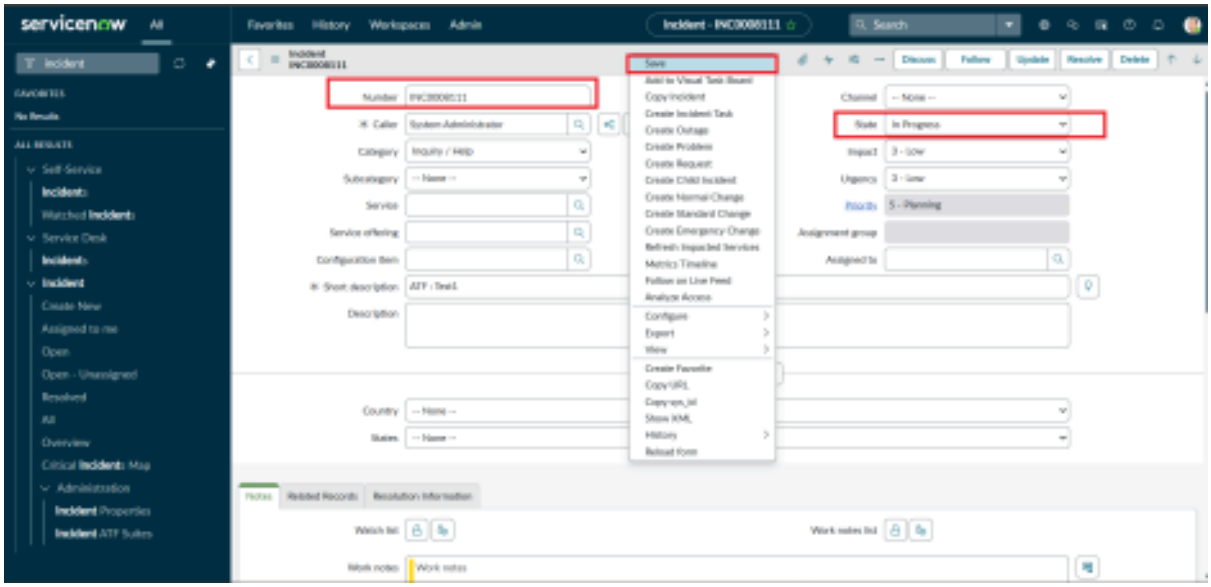

servicenow
All
Favorites
History
Workspaces
Admin
Incident - Create INC0010012
Search
incident
FAVORITES
No Results
ALL RESULTS
Self-Service
Incidents
Watched Incidents
Service Desk
Incidents
Incident
Create New
Assigned to me
Open
Open - Unassigned
Resolved
All
Overview
Critical Incidents Map
Administration
Incident Properties
New record
Submit
Resolve
Number
INC0010012
Caller
Adela Cervantiz
Category
Inquiry / Help
Subcategory
-- None --
Service
Service offering
Configuration item
Channel
-- None --
State
New
Impact
1 - High
Urgency
1 - High
Priority
1 - Critical
Assignment group
Software
Assigned to
Assigned To is mandatory for High impact incidents.
Short description
testing UI policy
Description
Related Search Results
Related Search
testing UI policy
Knowledge & Catalog (All
Company policies
No description
Whitelist IP
Need to whitelist an IP address?
Order


servicenow
Favorites
History
Workspaces
Admin
Incident - Create INC0010012
Search
Submit
Resolve
Number
INC0010012
Caller
Adela Cervantez
Category
Inquiry / Help
Subcategory
-- None --
Service
Service offering
Configuration item
Short description
testing UI policy
Description
Channel
State
New
Impact
1 - High
Urgency
1 - High
Priority
1 - Critical
Assignment group
Software
Assigned to
Beth Anglin
Related Search Results
Related Search
testing UI policy
Knowledge & Catalog (All
Company policies
No description
Whitelist IP
Need to whitelist an IP address?
Order
Firewall Rule Change
Cisco Firewall Appliance
Order


servicenow
Favorites
History
Workspaces
Admin
Incident - INC0010012
Search
Discuss
Follow
Update
Resolve
Delete
Number
INC0010012
Caller
Adela Cervantez
Category
Inquiry / Help
Subcategory
-- None --
Service
Service offering
Configuration item
Short description
testing UI policy
Description
Channel
State
New
Impact
2 - Medium
Urgency
3 - High
Priority
2 - High
Assignment group
Software
Assigned to
Related Search Results
Country
-- None --
States
-- None --
Notes
Related Records
Resolution Information


servicenow
Favorites
History
Workspaces
Admin
Incidents
Search
Number
Opened
Short description
Caller
Priority
State
Category
Assignment group
Assigned to
Updated
Updated by
Open
Open - Unassigned
Resolved
Create New
Assigned to me
All
Overview
Critical Incidents Map
Administration
Incident Properties
Incident ATF Suites





16. Conclusion

UI Policy is an important concept for creating dynamic, organized, and user-friendly applications. It allows an application to control interface elements based on specific conditions and user interactions. Fields can be displayed, hidden, made mandatory, disabled, or made read-only according to the requirements of the application.

The proper use of UI policies improves usability, reduces unnecessary information, minimizes data-entry errors, and helps users complete tasks efficiently. UI policies are especially useful in applications that contain complex forms with many fields.

A good UI policy should be simple, consistent, and easy for users to understand. It should be combined with proper validation, security, and backend business rules to create a reliable application. Therefore, UI policies play an important role in modern software development by connecting application logic with user interface behavior**.**